FACULTY OF ENGINEERING • UNIVERSITY OF RUHUNA

Department of Electrical and Information Engineering • Hapugala, Galle

COURSE PROJECT REPORT • EC4307 WEB APPLICATION DEVELOPMENT

SmartHall AI

An Intelligent University Hall Allocation, Campus Navigation, and Multimodal AI Permission Letter Verification System

AUTHOR / STUDENT NAME

D.M.S.N.H. Dissanayake (Reg: EG/2022/5574)

ACADEMIC CONTEXT

Faculty of Engineering, University of Ruhuna

REPOSITORY URL

github.com/Cognix-0/Smart-Hall-UOR-EFAC

TECHNOLOGY STACK

Node.js • Express 5 • EJS • MySQL 8 • Claude • Gemini

01. Executive Summary

SmartHall AI is an enterprise-grade university hall allocation and campus navigation web platform built specifically to resolve the logistical friction of venue scheduling at the Faculty of Engineering, University of Ruhuna (UOR-EFAC). Prior to SmartHall AI, batch representatives, society presidents, and faculty administration relied on disparate WhatsApp chats, static spreadsheets, and physical paper permission letters to schedule lectures and student society events. This produced recurring double-bookings, capacity mismatches, and days of administrative delay.

SmartHall AI establishes a unified, centralized digital architecture providing:

Algorithmic Auto-Allocation

Best-fit capacity placement solving bin-packing for lectures across all academic batches without hall clashes.

Multimodal AI Letter Check

Claude Opus checks scanned permission letters against 6 administrative approval criteria with instant decisioning.

PDF Timetable Ingestion

Python coordinate-based parsing that ingests official faculty timetable PDFs into live relational database records.

Natural Language Chatbot

Deterministic regex query reader + Gemini Flash fallback providing real-time room availability and walking directions.

Core Engineering Philosophy:

Zero bloated client-side JavaScript frameworks. SmartHall AI is built with server-side rendered Express 5 + EJS and tailored modern CSS. It achieves instant sub-100ms load times on low-bandwidth campus mobile networks and features a purposeful high-contrast dark theme designed for low-light lecture halls.

02. Project Idea & Motivation: Why It Matters

2.1 The Operational Bottleneck at UOR-EFAC

The Faculty of Engineering at Hapugala spans a large geographic footprint with diverse teaching venues:

Venue Code	Venue Name	Building Location	Capacity
MLT	Main Lecture Theatre	Administration Building, Ground floor	**250 seats**
LH1	Lecture Hall 1	Lecture Hall Complex, Ground floor	**180 seats**
LH2	Lecture Hall 2	Lecture Hall Complex, 1st floor	**130 seats**
ECC	Electrical Computer Centre	EIE Department, 2nd floor	**90 seats**
NCC	New Computer Centre	Computer Centre building, Ground floor	**120 seats**
OCC	Old Computer Centre	Computer Centre building, Ground floor	**80 seats**
DO	Drawing Office	Civil Engineering Building, 1st floor	**70 seats**
AUD	Faculty Auditorium	Faculty Auditorium, Main entrance	**400 seats**

2.2 Three Real-World Problem Statements

1. **Scheduling Collisions & Suboptimal Allocation:** Unofficial manual coordination led to simultaneous booking requests for the same venue. Additionally, small batches (e.g., 60 students) frequently occupied large venues (MLT, 250 seats), leaving major cohorts (180 students) stranded.
2. **Wayfinding & Navigation Deficits:** Junior students and incoming batches frequently missed the start of lectures because physical directions between departments were unindexed and tribal.
3. **Multi-Day Bureaucratic Delays:** Student societies (e.g., IEEE Student Branch) had to physically circulate paper permission letters for multiple signatures (President → Staff Counselor → Deputy Registrar), creating immense administrative drag.

03. Comprehensive CRUD Operations Breakdown

SmartHall AI enforces strict Role-Based Access Control (RBAC) across four user classes: **Admin**, **Batch Representative (Rep)**, **Society President**, and **Students / General Public**.

Entity	Create (C)	Read (R)	Update (U)	Delete (D)
Lecture Halls	`POST /halls` Name, code, building, floor, capacity, photo upload, directions.	`GET /halls` Grid search, seat filters, photo cards, live free/busy badge.	`POST /halls/:id` Update hall capacity, directions, or upload replacement photo.	`POST /halls/:id/delete` Cascades bookings; unlinks timetable allocations.
Batches	`POST /admin/batches` Name (e.g. "25"), enrolled student headcount.	`GET /admin/batches` Batches list with student headcounts.	`POST /admin/batches/:id` In-place editing of name and headcount.	`POST /admin/batches/:id/delete` Removes batch & cascades its lectures.
Time Slots	`POST /admin/slots` Start time, end time, label validation (`end > start`).	`GET /admin/slots` Daily teaching slots sequence.	`POST /admin/slots/:id` Adjust period timings with constraint validation.	`POST /admin/slots/:id/delete` Removes slot if no active lectures depend on it.
Timetable Lectures	`POST /timetable` or `/upload` Individual lecture or bulk CSV/PDF ingestion.	`GET /` & `/manage` Weekly grid, next lecture card, filter by batch.	`POST /timetable/:id` Change day, slot, subject, or assigned hall.	`POST /timetable/:id/delete` Removes lecture with custom modal confirmation.
Official PDF Timetables	`POST /timetable/upload` Ingests faculty PDF & maps coordinate matrix.	`GET /timetables/:file` Direct browser PDF view / public download.	`POST /timetable/upload` Replaces previous PDF for the batch.	`POST /timetable/files/:id/delete` Unlinks and unlinks file from storage.
Hall Bookings	`POST /bookings` Rep booking or Society booking with PDF letter upload.	`GET /bookings` Filterable bookings list, status indicators, letter preview.	`POST /bookings/:id/status` Admin approval, rejection, or revocation.	`POST /bookings/:id/delete` Cancels reservation and frees venue immediately.
Users & Access	`POST /admin/users` Bcrypt-hashed registration with role and batch binding.	`GET /admin/users` User directory with role indicators and batch assignments.	`POST /admin/users/:id` Role elevation, batch reassignment, password reset.	`POST /admin/users/:id/delete` Removes user without breaking booking audit logs.

3.1 Relational Database Integrity Guarantees

Rather than relying solely on application-layer logic, SmartHall AI delegates scheduling rules to MySQL 8.0 compound unique keys:

- `UNIQUE KEY one_class_per_batch (batch_id, day_of_week, time_slot_id)`: Mechanically impossible for a batch to have two scheduled classes simultaneously.

- `UNIQUE KEY one_class_per_hall (hall_id, day_of_week, time_slot_id)`: Physically impossible for two lectures to share the same hall in the same teaching block.
- `CHECK (end_at > start_at)`: Chronological sanity checks on all one-off events.

04. Intelligent Chatbot Architecture

SmartHall AI includes a dual-engine chatbot designed to assist students and reps who need immediate venue information from mobile devices without navigating complex tables.

Chatbot Dual-Engine Execution Pipeline

```
User Input: "Free halls tomorrow at 10am for 120 students"
                          ↓
[Stage 1: Regex Parser] → Extracted: { date: "2026-11-02", time: "10:00", count: 120 }
                          ↓
[Stage 2: Live SQL Check] → SELECT halls WHERE capacity ≥ 120 AND NOT IN (lectures ∪ bookings)
                          ↓
[Stage 3: Result Assembly] → "Free tomorrow at 10:00: Lecture Hall 1 (LH1, 180 seats) — LHC ground floor..."
```

4.1 Zero-Latency Deterministic Rule Parser

In `src/chatbot.js`, user prompts are parsed locally using optimized regex:

- **Temporal Resolution:** Understands `today`, `tomorrow`, ISO formats (`2026-11-02`), and natural expressions (`25th Oct`, `November 2`).
- **Time Resolution:** Normalizes 12-hour AM/PM formats (`10am`, `2:30pm`) and 24-hour military clock (`14:30`).
- **Capacity Extraction:** Detects student counts across multiple synonyms (e.g. `120 students`, `60 people`, `180 seats`, `pax`).

4.2 Gemini Flash LLM Fallback

When a student submits conversational or non-standard queries (e.g., *"Where is the Drawing Office?"* or *"Can you find a quiet spot for 40 students after lunch?"*), the system triggers the Gemini Flash API (`gemini-3.8-flash`). Hall knowledge is injected via strict system prompt bounding, preventing hallucinated venue names or non-existent rooms.

05. Advanced Engineering & Innovations

5.1 Algorithmic Best-Fit Auto-Allocation

Master timetable CSVs or PDFs often arrive without hall assignments. SmartHall AI's auto-allocation algorithm (`src/allocate.js`) treats hall allocation as a constrained optimization problem:

```
function allocate(entries, halls, taken) {
  // Sort entries: largest student cohorts placed first (hardest to fit)
  const sorted = entries.slice().sort((a, b) => b.student_count - a.student_count);
  const placed = [];
  const clashes = [];

  for (const entry of sorted) {
    // Find smallest available hall that accommodates batch headcount
    const fit = halls
      .filter((h) => h.capacity ≥ entry.student_count)
```

```
      .filter((h) => !taken.has(`${h.id}-${entry.day_of_week}-${entry.time_slot_id}`))
      .sort((a, b) => a.capacity - b.capacity)[0]; // Smallest adequate hall

    if (fit) {
      placed.push({ id: entry.id, hall_id: fit.id });
      taken.add(`${fit.id}-${entry.day_of_week}-${entry.time_slot_id}`);
    } else {
      clashes.push(entry);
    }
  }
  return { placed, clashes };
}
```

5.2 Automated PDF Timetable Ingestion Engine

The system includes an integrated Python parser (`src/parsePdf.py`) using `pypdf` :

- Extracts (x, y) coordinate bounding boxes of timetable cells on Page 1.
- Maps text into exact Monday–Friday day columns and 1-hour time blocks (`08:30-09:30` to `16:30-17:30`).
- Extracts course codes from Page 2 notes to convert shorthand codes (`EC5205`) into full official titles (*"Operating System and Network Programming"*).
- Maps university room codes (`LT1` → `LH1` , `CC` → `OCC`).

06. UI/UX Design Philosophy: Why the High-Contrast Dark UI?

A defining characteristic of SmartHall AI is its **purposeful high-contrast dark theme** (deep charcoal background `#0f1117`, elevated slate surface `#181b24`, university maroon `#7b1e33`, gold highlight `#f2b705`, and crisp emerald `#22c55e`).

> **Design Principle:**
> The visual presentation is not a decorative gimmick; it is an ergonomic engineering decision grounded in the physical conditions of the Faculty of Engineering campus.

6.1 Key Usability & Human-Computer Interaction (HCI) Factors

1. Low-Light Lecture Theatre Comfort

Lecture halls at UOR-EFAC are frequently darkened during projector slide presentations. A blazing white screen produces glare and pupil fatigue. The dark theme minimizes ocular strain during 2-hour lectures.

2. Outdoor High-Contrast Legibility

Galle campus outdoor corridors are brightly sunlit. By utilizing high-saturation accent colors (gold, busy-red, free-green) against deep slate, venue availability remains immediately legible under harsh sunlight.

3. OLED Battery Conservation

Over 80% of undergraduates browse schedules on mobile devices. True black and dark slate palettes substantially reduce power draw on modern OLED screens over a full 8-hour university day.

4. Physical Doorplate Metaphor

Venue badges (e.g. `LH1`, `MLT`, `AUD`) mimic tactile doorplates mounted on physical lecture room doors, creating an intuitive cognitive bridge between screen and physical space.

6.2 Custom Confirmation Modal UI

Rather than relying on browser-native `window.confirm()` popups (which are unstyled and interrupt the main thread), SmartHall AI implements an HTML5 `<dialog>` modal:

- **Dark Glass Backdrop:** Semi-transparent overlay with `backdrop-filter: blur(8px)`.
- **Contextual Danger Indicators:** Circular red badge with trash SVG icon and explicit deletion details (e.g., *"Are you sure you want to delete 'IS2101 Mathematics' for 27 (Mon 08:00–10:00)?"*).
- **Accessibility:** Full keyboard trapping, `Esc` key dismissal, and backdrop click-to-close.

07. Verification & Automated Test Suite

SmartHall AI includes a comprehensive Node.js native test suite (`node --test`) in `test/logic.test.js`:

Test Case Description	Target Component	Result
Campus board displays active event & upcoming lecture per hall	`src/board.js` (hallBoard)	PASS (0.32ms)
Next lecture correctly classifies "on now", "later today", "tomorrow"	`src/board.js` (nextLecture)	PASS (0.49ms)

Test Case Description	Target Component	Result
Chatbot natural language reader extracts date, time, and student count	`src/chatbot.js` (parseQuestion)	PASS (1.05ms)
Allocation selects smallest fitting hall and isolates unplaced clashes	`src/allocate.js` (allocate)	PASS (0.14ms)
Fixed lectures clash only on identical weekday and time slot	`src/clash.js` (fixedClashes)	PASS (0.09ms)

Test Suite Summary: 5 suites, 5 passing, 0 failing, 0 canceled. Execution duration: ~71ms.

Candidate Declaration:

I hereby confirm that this project and report represent original engineering work developed for the EC4307 Web Application Development module at the Faculty of Engineering, University of Ruhuna.

D.M.S.N.H. Dissanayake • EG/2022/5574

Department Endorsement:

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna

Module Coordinator • EC4307